Trabalho 2

Fundamentos de Processamento Paralelo e Distribuído

Matheus Darde Holdefer, Pedro Henrique Corral Livi,
Eduardo Castilhos de Castilho, Solano Schmidt Battisti
Escola Politécnica — PUCRS

14 de outubro de 2026

Resumo

Este relatório apresenta soluções em Go para dois problemas clássicos de concorrência: o Jantar dos Filósofos, proposto por Dijkstra em 1965, e o Produtor-Consumidor com buffer limitado. Para cada problema, analisamos a ocorrência de *deadlock* e *starvation* e coletamos métricas de desempenho (espera pelos garfos, vazão e ocupação do buffer), discutindo o efeito de cada estratégia.

Introdução

No Jantar dos Filósofos, N filósofos alternam entre pensar e comer, mas precisam de dois garfos para comer e existem apenas N garfos, um entre cada par de vizinhos. A solução deve evitar o *deadlock* (todos seguram o garfo da esquerda e esperam o da direita) e a *starvation* (um filósofo esperar indefinidamente). No Produtor-Consumidor, produtores e consumidores compartilham um buffer de tamanho fixo K: produtores esperam quando ele está cheio e consumidores quando está vazio.

Implementamos três versões do primeiro problema (`deadlock`, `hierarchy` e `tanenbaum`) e duas do segundo (`channel` e `semaphore`), descritas nas seções a seguir.

Metodologia. Os programas foram compilados com Go 1.22.2 e executados em Linux x86-64 com 1 vCPU. Cada configuração foi executada 10 vezes e as métricas impressas pelos programas foram extraídas por um script. Os valores são média $\pm$ desvio-padrão (dp), e as comparações usam o teste t de Welch.

1 Problema 1: Jantar dos Filósofos

1.1 Soluções implementadas

Cada filósofo repete R vezes o ciclo pensar, pegar os garfos, comer e soltar os garfos (pensar e comer duram de 67 a 267 ms). Cada garfo é uma goroutine com um canal sem buffer: o filósofo pega o garfo recebendo do canal e o devolve enviando, e a exclusão mútua decorre do próprio canal. O tempo entre ficar com fome e obter os dois garfos é registrado como **espera**.

Deadlock. O filósofo pega o garfo esquerdo, espera 2 s e só então pega o direito. Como pensar dura no máximo 267 ms, todos seguram o esquerdo antes de qualquer um tentar o direito, e o *deadlock* é determinístico. Um *watchdog* de 5 s detecta o travamento.

Hierarquia. Os garfos são numerados e cada filósofo i pega primeiro o de menor índice, $\min(i,\ (i+1) \bmod N)$, e depois o de maior índice. Assim, o filósofo $N-1$ pega o garfo 0 antes do garfo $N-1$, o que quebra a simetria da solução com *deadlock*.

Tanenbaum. Um mutex protege o vetor de estados (PENSANDO, FOME, COMENDO) e cada filósofo tem um semáforo binário. Um filósofo só passa a COMENDO se nenhum vizinho está comendo, e essa decisão é atômica (Algoritmo 1).

Algoritmo 1: Solução de Tanenbaum

```
1 Função Testar(i):
2 |   se estado[i] = FOME e estado[(i − 1) mod N] ≠ COMENDO e estado[(i + 1) mod N] ≠ COMENDO então
3 |   |   estado[i] ← COMENDO; sinalizar sem[i];
4 Função PegarGarfos(i):
5 |   travar mu; estado[i] ← FOME; Testar(i); liberar mu; esperar sem[i];
6 Função SoltarGarfos(i):
7 |   travar mu; estado[i] ← PENSANDO; Testar((i − 1) mod N); Testar((i + 1) mod N); liberar mu;
```

1.2 Deadlock e starvation

Tabela 1: Ocorrência de *deadlock* e *starvation* em cada versão.

Versão	Deadlock	Starvation
`deadlock`	Ocorre (10/10 execuções)	Não se aplica
`hierarchy`	Não ocorre (20/20 concluídas)	Não ocorre
`tanenbaum`	Não ocorre (20/20 concluídas)	Possível, não observada

Deadlock. As quatro condições de Coffman estão presentes: exclusão mútua (garfo de uso exclusivo), posse e espera (segura o esquerdo e espera o direito), não preempção (o garfo só é devolvido voluntariamente) e espera circular ($0 \to 1 \to \cdots \to 4 \to 0$). Nas 10/10 execuções o programa foi encerrado pelo *watchdog* em $\approx 5{,}01$ s.

Hierarquia. A ordem total dos garfos elimina a espera circular: quem espera pelo garfo j só segura garfos de índice menor, então a cadeia de espera termina em quem já tem o maior garfo. Como cada garfo é disputado por apenas dois filósofos e as filas dos canais de Go são FIFO, a espera é limitada e não há *starvation*. Essa garantia depende da FIFO do *runtime*.

Tanenbaum. Elimina a posse e espera, pois o filósofo só come se obtém os dois garfos de uma vez. Porém não há fila por ordem de chegada: vizinhos que alternem suas refeições podem manter um filósofo com fome indefinidamente. Com tempos aleatórios isso é improvável e não ocorreu em nenhuma das 20/20 execuções.

1.3 Análise de fairness

Como todos os filósofos completam as R refeições, comparamos a **espera média por refeição** de cada um, isto é, o tempo entre ficar com fome e obter os dois garfos. A Tabela 2 traz os dados para $R = 2$ e $R = 20$ refeições, com $N = 5$.

Tabela 2: Espera média por refeição (ms): média ± dp de 10 execuções.

	R = 2		R = 20	
Filósofo	**Hierarquia**	**Tanenbaum**	**Hierarquia**	**Tanenbaum**
Filósofo 0	157,5 ± 84,8	90,1 ± 60,4	138,3 ± 27,1	92,5 ± 25,4
Filósofo 1	101,3 ± 80,1	72,8 ± 46,0	124,5 ± 17,9	110,7 ± 13,8
Filósofo 2	130,8 ± 63,8	57,4 ± 67,4	112,5 ± 22,2	98,6 ± 19,3
Filósofo 3	72,8 ± 47,2	108,8 ± 73,9	87,2 ± 15,1	102,6 ± 28,2
Filósofo 4	88,0 ± 103,6	102,9 ± 81,9	141,5 ± 26,9	112,3 ± 31,8
Espera global	**110,1 ± 34,5**	**86,4 ± 24,2**	**120,8 ± 12,6**	**103,3 ± 4,6**

Com $R = 2$, cada filósofo faz apenas duas refeições e as médias variam muito de uma execução para outra, então os resultados não permitem apontar um algoritmo melhor. Com $R = 20$ os resultados ficam mais estáveis: Tanenbaum tem espera média menor que a hierarquia (103,3 ms contra 120,8 ms) e termina a execução um pouco antes (9,36 s contra 9,69 s). Isso acontece porque, na hierarquia, o filósofo pega um garfo de cada vez e pode ficar segurando o primeiro *sem comer* enquanto espera o segundo, o que bloqueia um vizinho que poderia estar comendo. Em Tanenbaum o filósofo só pega os garfos quando os dois estão livres, então nenhum garfo fica parado nas mãos de quem não está comendo.

Quanto à justiça entre os filósofos, a hierarquia trata alguns deles de forma diferente: os filósofos 4 e 0, que disputam o garfo 0 como primeira escolha, esperam mais (cerca de 141 ms e 138 ms) do que o filósofo 3 (cerca

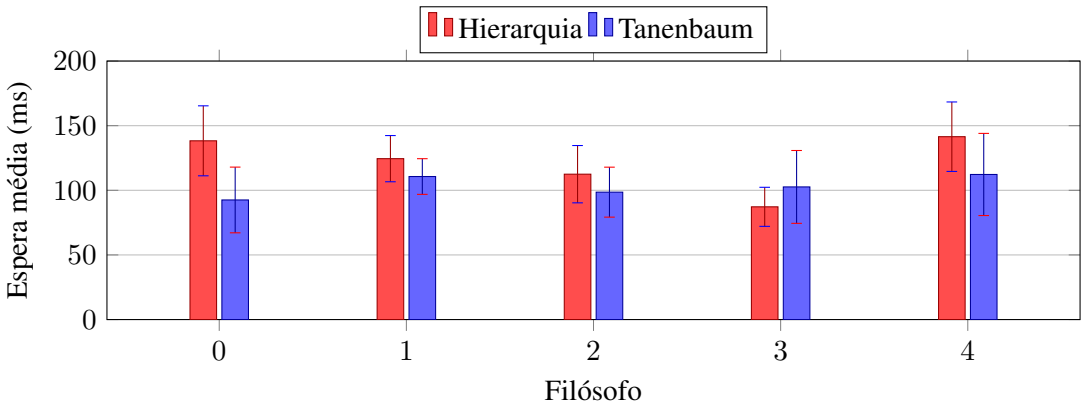


Figura 1: Espera média por refeição com $R = 20$ (barras de erro: dp entre execuções).

de 87 ms). Em Tanenbaum as esperas ficam mais parecidas entre si, entre 93 ms e 112 ms. Nas duas versões todos os filósofos completaram suas refeições, sem sinal de *starvation*.

Em resumo, nos nossos testes Tanenbaum teve espera média menor e distribuída de forma mais uniforme, enquanto a hierarquia é mais simples de implementar, mas favorece alguns filósofos em detrimento de outros.

1.4 Encerramento ordenado

Em todas as versões, cada filósofo executa exatamente R refeições e chama `wg.Done()`, e a `main` espera com `wg.Wait()` antes de imprimir o relatório. Nas versões `deadlock` e `hierarchy`, as goroutines dos garfos são laços infinitos e não são encerradas explicitamente: ficam bloqueadas e terminam junto com o processo. Em `tanenbaum` não há goroutines auxiliares. Na versão `deadlock`, um `select` compara o fim do `wg.Wait()` com um *timeout* de 5 s; se o *timeout* vence, o programa imprime o diagnóstico e encerra com `os.Exit(1)`.

2 Problema 2: Produtor-Consumidor

2.1 Soluções implementadas

Cada produtor gera 8 itens e os consumidores os processam, gastando de 10 a 50 ms por item. Medimos a vazão (itens consumidos por segundo), os itens por consumidor e a ocupação média do buffer (contador atômico amostrado a cada 2 ms). Variamos $K \in \{1, 2, 5, 10, 20, 50\}$ com 2 produtores e 2 consumidores (2P/2C, carga balanceada) e com 4 produtores e 2 consumidores (4P/2C, produção mais rápida que o consumo).

Channel. O buffer é um canal bufferizado de capacidade K: o produtor envia (`ch <- item`) e o consumidor recebe com `range ch`. O *runtime* bloqueia produtores com o canal cheio e consumidores com o canal vazio.

Semaphore. O buffer é circular e protegido por um mutex, usado só durante a inserção e a retirada. Dois semáforos contadores, implementados com canais bufferizados, controlam o acesso: `vagas` (inicia com K fichas) e `itens` (inicia com 0). O Algoritmo 2 mostra o protocolo, incluindo o encerramento dos consumidores.

2.2 Deadlock e starvation

Tabela 3: Ocorrência de *deadlock* e *starvation* em cada versão.

Versão	Deadlock	Starvation
`channel`	Não ocorre (120 concluídas)	Não ocorre
`semaphore`	Não ocorre (120 concluídas)	Não ocorre

Algoritmo 2: Produtor-consumidor com semáforos

```
1  Função Produtor(id):
2      para j ← 1 até 8 faça
3          produzir item (10–50 ms);
4          P(vagas); travar mu; inserir item em buf; liberar mu; V(itens);
5  Função ConsumirUm():
6      travar mu; item ← retirar de buf; liberar mu; V(vagas); processar item (10–50 ms);
7  Função Consumidor(id):
8      enquanto verdadeiro faça
9          se recebeu ficha de itens então
10             ConsumirUm();
11         senão se done está fechado então
12             enquanto há ficha em itens (sem bloquear) faça
13                 ConsumirUm();
14             retorna;
```

Channel. Não há espera circular: produtores só esperam consumidores (canal cheio) e consumidores só esperam produtores (canal vazio). O `close(ch)` ao fim da produção evita que os consumidores esperem para sempre. As filas do canal são FIFO e, em todas as execuções, os dois consumidores receberam itens (mínimo de 6 itens).

Semaphore. A ordem P(semáforo) seguida do mutex, mantido apenas na inserção e na retirada, evita o *deadlock* da ordem inversa: um consumidor com o mutex esperando um item que o produtor, esperando o mutex, produziria. A diferença entre os consumidores chegou a 6 itens em uma execução, mas nenhum ficou sem itens.

2.3 Efeito do tamanho do buffer

A Tabela 4 e a Figura 2 mostram vazão e ocupação média para cada valor de K.

Tabela 4: Vazão (itens/s) e ocupação média (itens) por K: média $\pm$ dp de 10 execuções.

	2P/2C				**4P/2C**			
	Vazão		Ocupação		Vazão		Ocupação	
K	Chan.	Sem.	Chan.	Sem.	Chan.	Sem.	Chan.	Sem.
1	50,4 ± 3,7	51,7 ± 4,1	0,39 ± 0,15	0,35 ± 0,16	62,0 ± 3,5	58,1 ± 2,2	0,85 ± 0,04	0,85 ± 0,03
2	51,6 ± 5,2	50,6 ± 3,9	0,63 ± 0,44	0,68 ± 0,33	62,3 ± 4,3	60,4 ± 6,0	1,64 ± 0,10	1,67 ± 0,08
5	51,8 ± 5,5	51,6 ± 4,2	0,45 ± 0,35	0,71 ± 0,50	61,1 ± 2,9	61,2 ± 5,1	3,77 ± 0,17	3,85 ± 0,17
10	55,1 ± 4,6	53,4 ± 4,8	0,77 ± 0,44	0,53 ± 0,28	60,0 ± 2,7	62,4 ± 5,0	6,12 ± 0,44	5,66 ± 0,75
20	53,7 ± 2,0	51,5 ± 5,1	0,43 ± 0,32	0,74 ± 0,55	61,4 ± 3,8	62,1 ± 5,0	7,08 ± 1,39	7,40 ± 1,55
50	52,9 ± 4,4	52,8 ± 3,0	0,43 ± 0,28	0,62 ± 0,44	62,1 ± 5,8	60,6 ± 3,2	6,55 ± 1,11	7,44 ± 0,67

Carga balanceada (2P/2C). A vazão fica em 50–55 itens/s e o tempo total em 292–319 ms para qualquer K; as diferenças entre valores de K têm o tamanho do desvio entre execuções, causado pelo sorteio dos tempos de trabalho. A ocupação média fica abaixo de 1 item (0,35–0,77): como produção e consumo têm o mesmo ritmo, o buffer quase nunca acumula.

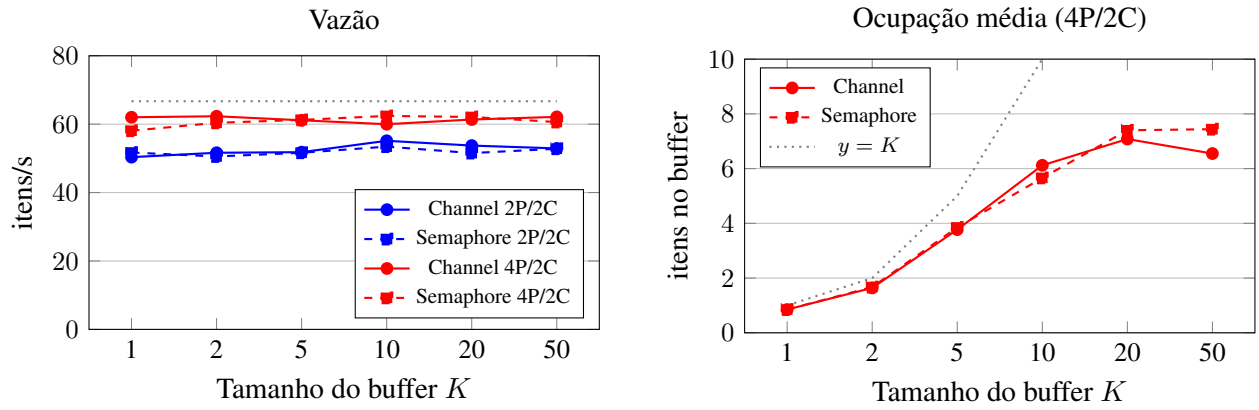


Figura 2: Vazão e ocupação média em função de K. A linha pontilhada da vazão marca o limite teórico de 2 consumidores ($2/30$ ms $\approx$ 66,7 itens/s).

Produtores mais rápidos (4P/2C). Aqui o gargalo é o consumo: a vazão fica em torno de 60 itens/s, perto do máximo que dois consumidores conseguem processar (linha pontilhada da Figura 2), e quase não muda com K. O que muda é a ocupação do buffer. Com K pequeno ele está quase sempre cheio e os produtores ficam bloqueados esperando vaga. Com K maior ele passa a acumular itens, mas a ocupação para de crescer por volta de $K = 20$ (cerca de 7 itens), porque a produção termina antes que o acúmulo chegue a preencher o buffer. Ou seja, um buffer maior não aumenta a vazão quando o consumo é o gargalo: ele só guarda os itens que os produtores já geraram e ajuda a absorver picos de produção.

Channel contra semaphore. As duas versões tiveram desempenho praticamente igual, tanto na vazão quanto na ocupação do buffer. Isso é esperado, pois os semáforos da nossa implementação também são canais, e o custo extra do mutex e das operações adicionais é muito pequeno perto dos 10 a 50 ms que cada item leva para ser produzido ou consumido. A única diferença visível aparece com $K = 1$ e 4P/2C, em que a versão com semáforos foi um pouco mais lenta (cerca de 6%), pois um buffer de uma posição obriga produtores e consumidores a se sincronizarem a cada item. Como o desempenho é equivalente, a versão com `channel` é preferível por ser mais simples e menos sujeita a erros.

2.4 Encerramento ordenado

Channel. Cada produtor chama `wg.Done()` ao terminar. Uma goroutine auxiliar espera todos os produtores e então fecha o canal (`close(ch)`). Os consumidores usam `range ch`, que após o `close` ainda entrega os itens que restam no buffer e só então termina: nenhum item é perdido.

Semaphore. Os semáforos não são fechados. Uma goroutine fecha o canal `done` depois que todos os produtores terminam. Como cada produtor sinaliza `itens` antes de `wg.Done()`, quando `done` fecha o número de fichas é exatamente o número de itens restantes. Então cada consumidor drena o buffer com um `select` não bloqueante (`default`) e retorna quando `itens` esvazia.

Nas duas versões, a `main` espera os consumidores, para a goroutine que mede a ocupação e só então imprime o relatório.

3 Uso de ferramentas de IA

Foi utilizado o Claude (Anthropic) na **redação do relatório** para gerar as tabelas, os gráficos e os pseudo-códigos a partir dos códigos e das saídas dos programas, além da revisão ortográfica do texto. Na **criação do código** o Claude foi utilizado *apenas* para a organização dos outputs do programa.

Conclusões

Os experimentos confirmam que quebrar uma das condições de Coffman elimina o *deadlock*: a hierarquia de recursos impede a espera circular e a solução de Tanenbaum impede a posse e espera. Tanenbaum teve menor espera média e maior justiça entre os filósofos, enquanto a hierarquia é assimétrica, mas tem espera limitada por construção. No Produtor-Consumidor, o tamanho do buffer só altera a ocupação quando a produção excede o consumo, e a vazão continua limitada pelo gargalo. Channel e semáforos tiveram desempenho equivalente, sendo o `channel` a opção mais simples.